

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ІІІ

Звіт

з лабораторної роботи №2 з дисципліни
«Системне програмне забезпечення»

„Засоби обміну даними”

Виконав ІІ-45 Янов Б. Є.

Перевірив Бабич Б. Б.

Київ 2026

Лабораторна робота №2

Засоби обміну даними

Завдання:

1. Написати програму з використанням 2-х процедур:

- Процедура введення і перетворення цілого числа. Після цього треба виконати математичну дію над числом (номер завдання 19).
- Процедура переведення отриманого результату в рядок та виведення його на екран.

2. Програма повинна мати захист від некоректного введення вхідних даних (символи, переповнення, ділення на 0 і т.і.)

Текст програми

```
.model small
.stack 100h

.data
    msg_input db 'Enter an integer: $'
    ; crlf
    msg_res   db 13, 10, 'Result (-34): $'
    msg_err   db 13, 10, 'Error: Invalid input! (Numbers only)$'
    msg_ovf   db 13, 10, 'Error: Overflow/Underflow! Keep it 16-bit.$'

    ; Buf for 10th DOS function: max len, real len, char array
    buffer    db 7, 0, 7 dup(0)
    num       dw 0
    is_neg    db 0

.code
main proc

    mov ax, @data
    mov ds, ax

    lea dx, msg_input
    mov ah, 09h
    int 21h

    call InputProc
    jc  exit_prog          ; Carry Flag = 1 => error

    ; Var. 19: -34
    mov ax, num
    sub ax, 34
    jo  overflow_err

    push ax                ; save ax
    lea dx, msg_res
```

```

        mov     ah, 09h
        int     21h
        pop     ax                ; restore ax

        call    OutputProc
        jmp     exit_prog

overflow_err:
        lea     dx, msg_ovf
        mov     ah, 09h
        int     21h
        jmp     exit_prog

exit_prog:
        mov     ax, 4c00h
        int     21h

main endp

InputProc proc

        lea     dx, buffer
        mov     ah, 0Ah
        int     21h

        lea     si, buffer+2      ; ptr of symbols
        mov     cl, buffer+1      ; num of symbols
        xor     ch, ch
        cmp     cx, 0
        je      input_error       ; zero characters

        xor     bx, bx            ; num accumulator
        mov     is_neg, 0

        ; check for sign
        mov     al, [si]
        cmp     al, '-'
        jne     check_plus
        mov     is_neg, 1
        inc     si
        dec     cx
        jmp     parse_loop

check_plus:
        cmp     al, '+'
        jne     parse_loop
        inc     si
        dec     cx

parse_loop:

```

```

        cmp    cx, 0
        je     parse_done

        mov    al, [si]
        ; check if it is a number (0-9)
        cmp    al, '0'
        jl     input_error
        cmp    al, '9'
        jg     input_error

        sub    al, '0'           ; char -> num
        xor    ah, ah
        push   cx               ; save counter

        mov    cx, ax           ; cx = current num
        mov    ax, bx
        mov    dx, 10
        imul   dx               ; ax = ax * 10
        jo     ovf_error

        add    ax, cx           ; ax = ax + current num
        jo     ovf_error

        mov    bx, ax
        pop    cx               ; restore counter

        inc    si
        dec    cx
        jmp    parse_loop

parse_done:
        cmp    is_neg, 1
        jne    save_number
        neg    bx               ; swap left bit -> num is
positive
        jo     ovf_error

        save_number:
        mov    num, bx
        cld                     ; Carry Flag = 0
        ret

        ovf_error:
        pop    cx               ; clear stack when exiting
loop
        lea    dx, msg_ovf
        mov    ah, 09h
        int    21h
        stc                     ; Carry Flag = 1
        ret

```

```

input_error:
    lea    dx, msg_err
    mov    ah, 09h
    int    21h
    stc
    ret

InputProc endp

OutputProc proc
    cmp    ax, 0
    jge    start_convert    ; skip - if ax >= 0

    ; print -
    push ax
    mov    ah, 02h
    mov    dl, '-'
    int    21h
    pop    ax
    neg    ax                ; swap left bit

start_convert:
    xor     cx, cx            ; reset cx
    mov     bx, 10

divide_loop:
    xor     dx, dx
    div     bx                ; ax = ax / 10, dx = rem
    push    dx                ; push to stack
    inc     cx
    test    ax, ax
    jnz     divide_loop

print_loop:
    pop     dx
    add     dl, '0'           ; convert num to char
    mov     ah, 02h
    int     21h
    loop    print_loop

    ret

OutputProc endp

end main

```

Вміст .lst файлу

```

1    0000                                .model small
2    0000                                .stack 100h
3
4    0000                                .data
5    0000  45 6E 74 65 72 20 61+  msg_input db 'Enter an
integer: $'
6          6E 20 69 6E 74 65 67+
7          65 72 3A 20 24
8                                ; crlf
9    0013  0D 0A 52 65 73 75 6C+  msg_res  db 13, 10,
'Result (-34): $'
10         74 20 28 2D 33 34 29+
11         3A 20 24
12    0024  0D 0A 45 72 72 6F 72+  msg_err  db 13, 10,
'Error: Invalid input! (Numbers only)$'
13         3A 20 49 6E 76 61 6C+
14         69 64 20 69 6E 70 75+
15         74 21 20 28 4E 75 6D+
16         62 65 72 73 20 6F 6E+
17         6C 79 29 24
18    004B  0D 0A 45 72 72 6F 72+  msg_ovf  db 13, 10,
'Error: Overflow/Underflow! Keep it 16-bit.$'
19         3A 20 4F 76 65 72 66+
20         6C 6F 77 2F 55 6E 64+
21         65 72 66 6C 6F 77 21+
22         20 4B 65 65 70 20 69+
23         74 20 31 36 2D 62 69+
24         74 2E 24
25
26                                ; Buf for 10th      DOS function: max
len, real len, char array
27    0078  07 00 07*(00)              buffer      db 7, 0, 7 dup(0)
28    0081  0000                        num          dw 0

```

```

29    0083    00                is_neg        db 0
30
31    0084                .code
32    0000                main proc
33    0000    B8 0000s                mov  ax,      @data
34    0003    8E D8                mov  ds,      ax
35
36    0005    BA 0000r                lea  dx,
msg_input
37    0008    B4 09                mov  ah,      09h
38    000A    CD 21                int  21h
39
40    000C    E8 0028                call InputProc
41    000F    72 21                jc   exit_prog
; Carry Flag = 1 => error
42
43                                ; Var. 19: -34
44    0011    A1 0081r                mov  ax,      num
45    0014    2D 0022                sub  ax,      34
46    0017    70 0F                jo   overflow_err
47
48    0019    50                push ax                ; save
ax
49    001A    BA 0013r                lea  dx,      msg_res
50    001D    B4 09                mov  ah,      09h
51    001F    CD 21                int  21h
52    0021    58                pop  ax                ;
restore ax
53
54    0022    E8 0092                call OutputProc
55    0025    EB 0B 90                jmp  exit_prog
56
57    0028                overflow_err:

```

2

LAB.ASM

```
58 0028 BA 004Br      lea dx,      msg_ovf
59 002B B4 09        mov ah,      09h
60 002D CD 21        int 21h
61 002F EB 01 90      jmp exit_prog
62
63 0032                exit_prog:
64 0032 B8 4C00       mov ax,      4c00h
65 0035 CD 21        int 21h
66 0037                main endp
67
68
69
70 0037                InputProc proc
71 0037 BA 0078r      lea dx,      buffer
72 003A B4 0A        mov ah,      0Ah
73 003C CD 21        int 21h
74
75 003E BE 007Ar      lea si,
buffer+2          ; ptr of symbols
76 0041 8A 0E 0079r  mov cl,
buffer+1          ; num of symbols
77 0045 32 ED        xor ch,      ch
78 0047 83 F9 00     cmp cx,      0
79 004A 74 62        je  input_error
                ; zero characters
80
81 004C 33 DB        xor bx,      bx
                ; num accumulator
82 004E C6 06 0083r 00      mov is_neg, 0
83
```

```

84                                     ; check for sign
85    0053    8A 04                     mov  al,      [si]
86    0055    3C 2D                     cmp  al,      '-'
87    0057    75 0A                     jne  check_plus
88    0059    C6 06 0083r 01            mov  is_neg, 1
89    005E    46                         inc  si
90    005F    49                         dec  cx
91    0060    EB 07 90                   jmp  parse_loop
92
93    0063                                check_plus:
94    0063    3C 2B                     cmp  al,      '+'
95    0065    75 02                     jne  parse_loop
96    0067    46                         inc  si
97    0068    49                         dec  cx
98
99    0069                                parse_loop:
100   0069    83 F9 00                   cmp  cx,      0
101   006C    74 25                       je  parse_done
102
103   006E    8A 04                     mov  al,      [si]
104                                     ; check if it is a number
(0-9)
105   0070    3C 30                     cmp  al,      '0'
106   0072    7C 3A                     jl  input_error
107   0074    3C 39                     cmp  al,      '9'
108   0076    7F 36                     jg  input_error
109
110   0078    2C 30                     sub  al,      '0'
      ; char -> num
111   007A    32 E4                     xor  ah,      ah
112   007C    51                         push cx      ; save
counter
113
114   007D    8B C8                     mov  cx,      ax
      ; cx = current num

```

3

LAB.ASM

```
115 007F 8B C3          mov ax,      bx
116 0081 BA 000A        mov dx,      10
117 0084 F7 EA          imul dx          ;
ax = ax * 10
118 0086 70 1C          jo  ovf_error
119
120 0088 03 C1          add ax,      cx
      ; ax = ax + current num
121 008A 70 18          jo  ovf_error
122
123 008C 8B D8          mov bx,      ax
124 008E 59             pop cx          ;
restore counter
125
126 008F 46             inc si
127 0090 49             dec cx
128 0091 EB D6          jmp  parse_loop
129
130 0093                parse_done:
131 0093 80 3E 0083r 01      cmp  is_neg, 1
132 0098 75 04          jne  save_number
133 009A F7 DB          neg  bx          ;
swap left bit ->      num is positive
134 009C 70 06          jo  ovf_error
135
136 009E                save_number:
137 009E 89 1E 0081r        mov  num, bx
138 00A2 F8             cld          ; Carry
Flag = 0
139 00A3 C3             ret
```

```

140
141      00A4                      ovf_error:
142      00A4  59                      pop  cx          ; clear
stack when      exiting  loop
143      00A5  BA 004Br                      lea  dx,      msg_ovf
144      00A8  B4 09                      mov  ah,      09h
145      00AA  CD 21                      int  21h
146      00AC  F9                      stc          ; Carry
Flag = 1
147      00AD  C3                      ret
148
149      00AE                      input_error:
150      00AE  BA 0024r                      lea  dx,      msg_err
151      00B1  B4 09                      mov  ah,      09h
152      00B3  CD 21                      int  21h
153      00B5  F9                      stc
154      00B6  C3                      ret
155      00B7                      InputProc endp
156
157
158
159      00B7                      OutputProc      proc
160      00B7  3D 0000                      cmp  ax,      0
161      00BA  7D 0A                      jge
start_convert    ; skip - if ax >= 0
162
163                      ; print -
164      00BC  50                      push ax
165      00BD  B4 02                      mov  ah,      02h
166      00BF  B2 2D                      mov  dl,      '-'
167      00C1  CD 21                      int  21h
168      00C3  58                      pop  ax
169      00C4  F7 D8                      neg  ax          ;
swap left bit
170
171      00C6                      start_convert:

```

4

LAB.ASM

```
172  00C6  33 C9                xor  cx,      cx
      ; reset cx
173  00C8  BB 000A              mov  bx,      10
174  00CB                                divide_loop:
175  00CB  33 D2                xor  dx,      dx
176  00CD  F7 F3                div  bx              ;
ax = ax / 10, dx      = rem
177  00CF  52                                push dx          ; push
to stack
178  00D0  41                                inc  cx
179  00D1  85 C0                test ax,      ax
180  00D3  75 F6                jnz  divide_loop
181
182  00D5                                print_loop:
183  00D5  5A                                pop  dx
184  00D6  80 C2 30              add  dl,      '0'
      ; convert num to char
185  00D9  B4 02                mov  ah,      02h
186  00DB  CD 21                int  21h
187  00DD  E2 F6                loop print_loop
188
189  00DF  C3                                ret
190  00E0                                OutputProc  endp
191
192                                end main
```

5

Symbol Table

Symbol Name	Type	Value
??DATE	Text	"05/23/26"
??FILENAME	Text	"LAB"
??TIME	Text	"22:47:37"
??VERSION	Number	0400
@32BIT	Text	0
@CODE	Text	_TEXT
@CODESIZE	Text	0
@CPU	Text	0101H
@CURSEG	Text	_TEXT
@DATA	Text	DGROUP
@DATASIZE	Text	0
@FILENAME	Text	LAB
@INTERFACE	Text	00H
@MODEL	Text	2
@STACK	Text	DGROUP
@WORDSIZE	Text	2
BUFFER	Byte	DGROUP:0078
CHECK_PLUS	Near	_TEXT:0063
DIVIDE_LOOP	Near	_TEXT:00CB
EXIT_PROG	Near	_TEXT:0032
INPUTPROC	Near	_TEXT:0037
INPUT_ERROR	Near	_TEXT:00AE
IS_NEG	Byte	DGROUP:0083
MAIN	Near	_TEXT:0000
MSG_ERR	Byte	DGROUP:0024
MSG_INPUT	Byte	DGROUP:0000
MSG_OVF	Byte	DGROUP:004B

MSG_RES	Byte	DGROUP:0013
NUM	Word	DGROUP:0081
OUTPUTPROC	Near	_TEXT:00B7
OVERFLOW_ERR	Near	_TEXT:0028
OVF_ERROR	Near	_TEXT:00A4
PARSE_DONE	Near	_TEXT:0093
PARSE_LOOP	Near	_TEXT:0069
PRINT_LOOP	Near	_TEXT:00D5
SAVE_NUMBER	Near	_TEXT:009E
START_CONVERT	Near	_TEXT:00C6

Groups & Segments	Bit	Size	Align	Combine	Class
-------------------	-----	------	-------	---------	-------

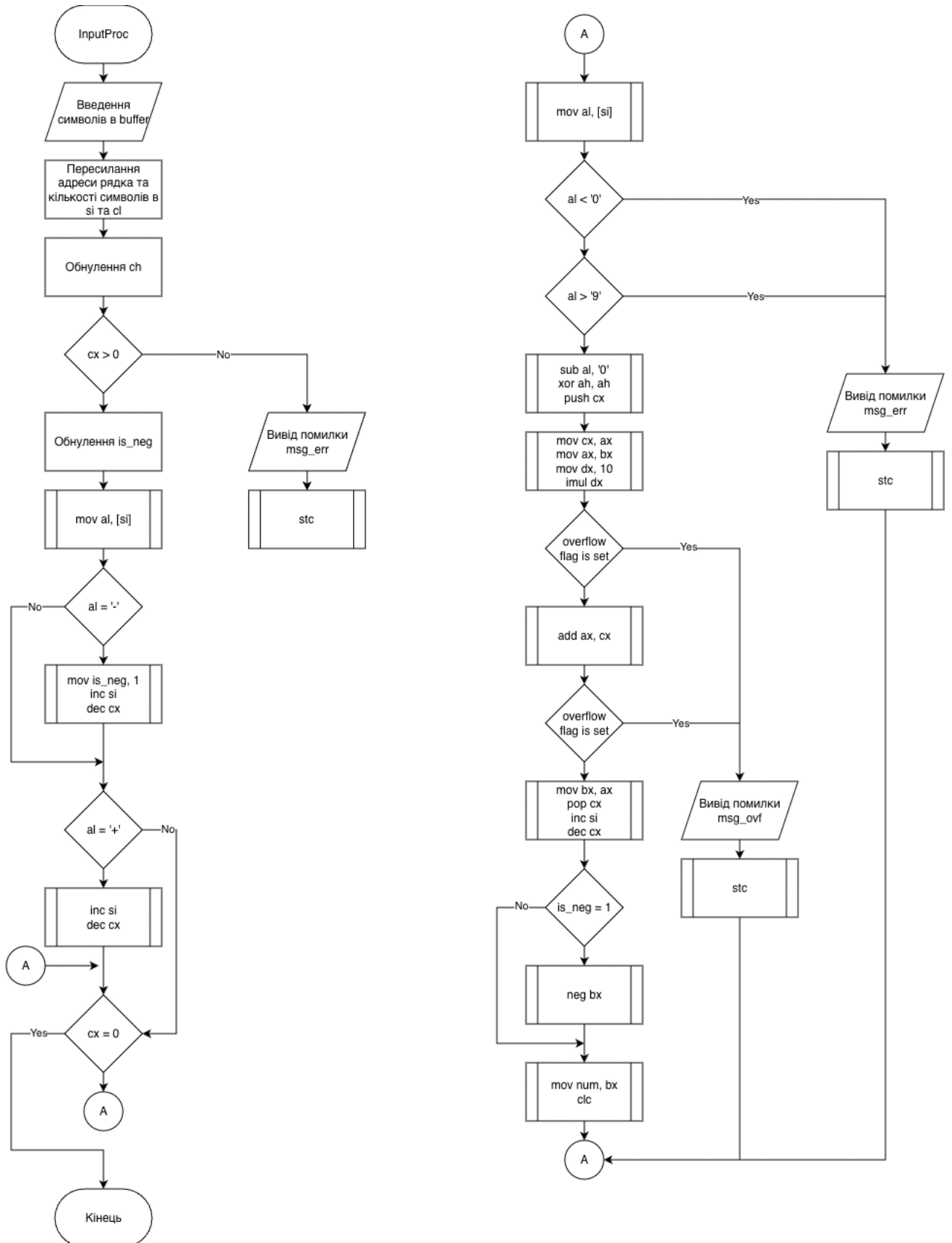
DGROUP	Group				
STACK	16	0100	Para	Stack	STACK
_DATA	16	0084	Word	Public	DATA
_TEXT	16	00E0	Word	Public	CODE

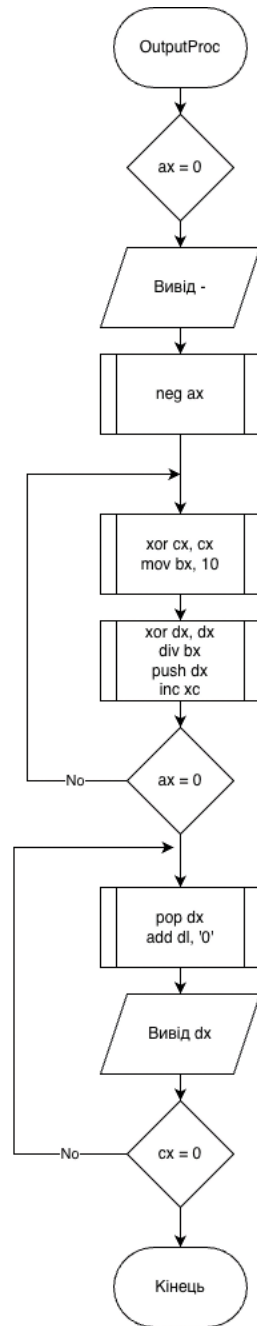
Вміст .map файлу

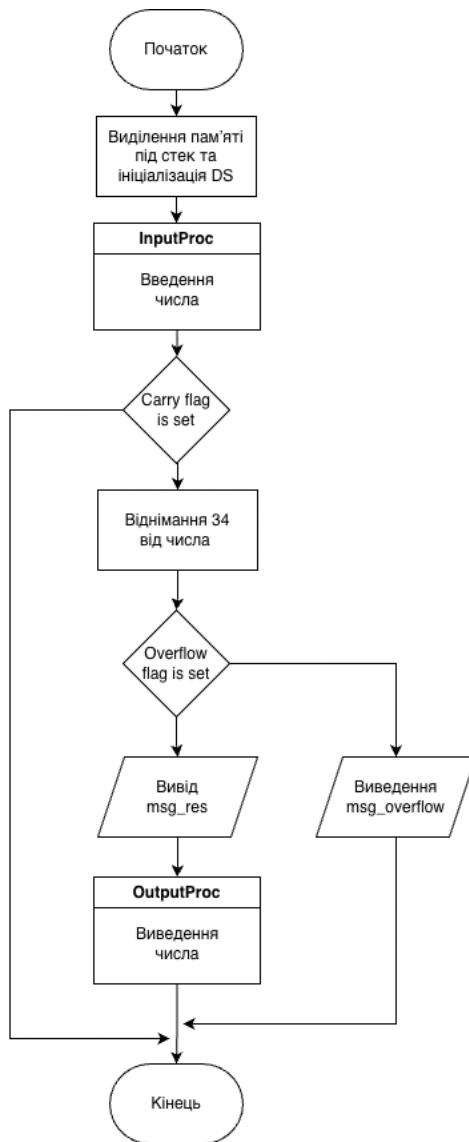
Start	Stop	Length	Name	Class
00000H	000DFH	000E0H	_TEXT	CODE
000E0H	00163H	00084H	_DATA	DATA
00170H	0026FH	00100H	STACK	STACK

Program entry point at 0000:0000

Блок схема програми







Приклад роботи програми

```

C:\LAB2>LAB.EXE
Enter an integer: 1488
Result (-34): 1454
C:\LAB2>
  
```

Висновок

1. Реалізовано програму з 2 процедурами для введенні і перетворення цілого числа та виведення цілого числа.
2. Реалізовано виконання математичної дії над числом.
3. Програма має захист від некоректного введення вхідних даних.